

AI Resume Screening System

Assignment II: Production Refactoring, API Design, and Quality Assurance

AIMLCZG546 - Software Engineering for Machine Learning
Group No. 179

Group details

BITS ID	Name	Contribution	Percentage
2025AB05113	Prashant	ML implementation and API, Architecture and code quality	
2025AA05032	Prathap Wagle	ML implementation and API, Architecture and code quality	
2024AC05999	Prasanna R T	Test design, execution and review	
2024AC05914	Pranav Mehrotra	QA evidence and final review	

Submission statement. This submission evolves the working Assignment I system without removing its upload, explanation, human-oversight, or audit features.

Assignment compliance mapping

Requirement	Implementation	Verified evidence
1. Modular refactoring	ML, quality, shared application, audit, API, and UI modules	ml/, quality/, services/, app/, streamlit_app.py
2. Research vs production	Inline notebook prototype compared with production clean_text	179.ipynb, Section 3
3. Logging and errors	INFO/WARNING/ERROR across six critical modules	logging_config.py, logs/
4. Formatting and linting	Recorded initial findings, then Black/isort/Flake8 pass	reports/lint_before.txt and lint_after.txt
5. REST API	Startup model load; typed prediction, health, and metrics	app/api.py, app/schemas.py
6-7. Tests and ML tests	Unit, data, loss, persistence, inference, continuity, API	35 tests; reports/pytest_report.txt
8. Quality metrics	Accuracy, F1, Brier, top-3, schema, balance, drift helper	reports/metrics_snapshot.json
9. Production and security	Shadow/canary strategy and resume-data controls	Section 9

1. Continuity from Assignment I

Assignment II uses the same HR/recruitment problem, labelled job-role data, TF-IDF and logistic-regression pipeline, keyword explanation, human decision note, and measurable quality goals developed in Assignment I. The monolithic prototype is refactored into production modules and exposed through FastAPI.

Assignment I capability	Preserved Assignment II component
TXT/PDF/DOCX resume upload	ml/ingestion.py
Text normalization and classification	ml/preprocessing.py, ml/trainer.py
Explanation and human oversight	services/scoring.py
Consistent submission orchestration	services/application.py
Audit history without raw resume storage	services/audit.py
Recruiter-facing dashboard	streamlit_app.py
New production inference contract	app/api.py and app/schemas.py

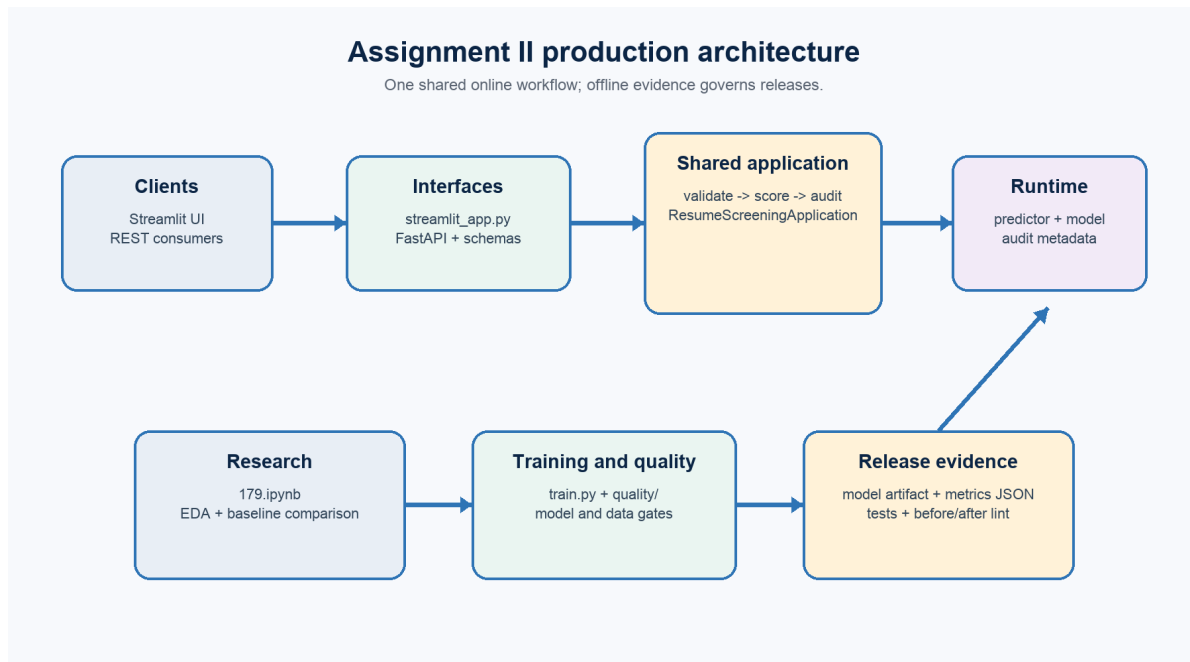


Figure 1. Shared online workflow and evidence-governed offline release.

2. Refactoring with OOP and functional principles

Data and validation. `ml/data.py` creates the labelled dataset and returns an immutable `DataQualityReport` after schema, missing-value, duplicate, and class coverage checks.

Ingestion. `ml/ingestion.py` isolates TXT/PDF/DOCX parsing, file-type checks, empty-file checks, and a 5 MB upload limit.

Training. `ml/trainer.py` constructs the reusable scikit-learn pipeline, measures validation metrics, enforces release gates, retrains on all data, and persists a versionable artifact.

Inference and services. `ModelPredictor` owns artifact loading and stable probability output. `ResumeScoringService` adds explanation and human oversight without coupling those rules to the classifier.

Application boundaries. `ResumeScreeningApplication` coordinates `validate`, `score`, and `audit` for both FastAPI and Streamlit. `AuditRepository` stores only metadata.

Quality and configuration. `config.py` centralizes runtime paths and versioning; `quality/` computes model/data metrics and drift signals; training persists metrics JSON.

3. Research code vs production code

The canonical `179.ipynb` now demonstrates the same preprocessing component twice. The first version is deliberately notebook-oriented; the second imports the tested production function. It also records class/text-length EDA, compares MultinomialNB with logistic regression on one fixed split, and displays a classification report and confusion matrix before the production hand-off.

```
# Research prototype in 179.ipynb
def prototype_clean_text(text):
    return re.sub(r"\s+", " ", re.sub(r"[^a-z0-9+.#. ]", " ", text.lower())).strip()

# Production use
from ml.preprocessing import clean_text
clean_text(" Python TESTING!! ")
```

Aspect	Research prototype	Production component
Purpose	Rapid feasibility	Repeatable service behavior
Structure	Inline notebook cell	Importable typed function
Validation	Assumes string input	Type, empty, size, semantic checks
Observability	Displayed output	Structured INFO/WARNING/ERROR logs
Quality evidence	Manual inspection	Unit and API integration tests

4. Error handling and logging

Logging is configured centrally and writes to console plus a rotating project-relative file (1 MB, three backups). Raw resume text is neither logged nor stored in the audit CSV.

Critical module	INFO	WARNING / ERROR
ml/preprocessing.py	Input/output character counts	Wrong type, empty, oversized, or unusable content
ml/ingestion.py	Type and extracted length	Unsupported, empty, oversized, or failed extraction
ml/data.py	Rows, schema, quality report	Missing schema/values or insufficient classes
ml/trainer.py	Training rows, artifact, metrics	Fit exception or failed quality gate
ml/predictor.py	Artifact and prediction metadata	Missing artifact or inference exception
app/api.py	Startup model load and successful requests	Invalid request, unavailable model, unexpected error

5. Code formatting and linting

The initial review established a genuine baseline before remediation. The configured tools were then applied to the complete Python codebase and rerun after implementation changes.

```
BEFORE (2026-07-26)
Black: 15 files needed formatting.
isort: 6 files had incorrectly sorted imports.
Flake8: 29 violations.

AFTER (2026-08-01)
$ black --check . -> PASS
$ isort --check-only .
PASS - no incorrectly sorted imports.

$ flake8 .
PASS - no violations.
```

Evidence 1. Recorded baseline and reproducible post-remediation results.

6. REST API design

Endpoint	Contract	Status behavior
GET /health	Typed status, model_loaded, and API version	200 ready; 503 artifact missing
POST /predict and /v1/predictions	Typed candidate and resume request; typed ranking response	200 success; 422 invalid; 503 model; 500 unexpected
GET /metrics	Persisted model/data release snapshot	200 available; 404 snapshot missing
GET /docs	Generated OpenAPI/Swagger documentation	Interactive contract documentation

Validation occurs at both schema and domain layers. A punctuation-only resume is rejected after normalization, blank candidate names are rejected, model loading occurs at startup, and readiness fails when the artifact is missing.

```
POST /v1/predictions -> 200
{candidate_name, predicted_role, confidence,
 role_ranking[6], explanation, decision_note}

GET /health with missing artifact -> 503
```

7. Testing strategy and implementation

Type	Representative checks	Evidence
Unit	Normalization, invalid content, TXT ingestion, audit privacy	test_preprocessing.py, test_continuity.py
Data validation	Schema, missing values, duplicates, class coverage	test_data_quality.py
Model training	Loss decreases, gates, determinism, persistence round-trip	test_training.py
Model inference	Ranking/probability contract, direction, invariance, empty input	test_inference.py
API integration	Lifecycle, aliases, metrics, audit privacy, validation/errors	test_api.py
Quality monitoring	Data summary and text-length drift detection	test_quality_metrics.py

```
$ pytest -q
..... [100%]
35 passed in 0.51s
```

Evidence 2. Automated unit, ML-specific, data, continuity, and API tests.

8. Model and data quality metrics

Category	Metric	Measured	Gate / interpretation
Model	Accuracy	1.000	Required ≥ 0.800
Model	Weighted F1	1.000	Required ≥ 0.800
Model	Top-3 accuracy	1.000	Monitoring metric
Model	Multiclass Brier	0.725	Lower is better; monitored
Data	Schema validity	True	Required columns present
Data	Missing-value rate	0.000	Required = 0
Data	Missing required values	0	Required = 0
Data	Duplicate rows	0	Monitored for evaluation bias
Data	Minority-class fraction	0.167	Balanced 6-class set

The evaluation uses 12 stratified validation rows from a deliberately small 24-row synthetic dataset. The results prove that measurement and release gates are implemented; they are not production-performance claims. A reusable text-length drift check flags batches beyond three reference standard deviations.

9. Testing in production and security

9.1 Shadow and canary strategy

A candidate model would receive copies of production requests while the current model remains recruiter-visible. The team would compare latency, error rate, class distribution, confidence, disagreement, and manually reviewed correctness. Promotion would require predefined gates, followed by a small canary with automatic rollback thresholds.

9.2 Security and privacy

Resume content is sensitive personal data. The implementation enforces type, length, semantic-content, file-type, and upload-size validation; does not log or persist raw resume text; and separates audit metadata. Operational monitoring should also detect keyword stuffing, input length drift, confidence shifts, and unexpected role distributions. Production deployment additionally requires TLS, authentication, rate limiting, malware scanning, isolated parsing, encryption, retention limits, and least-privilege access to data and model artifacts.

10. Reproducible execution

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python train.py
pytest -q
black --check .
isort --check-only .
flake8 .
python run_api.py
# API docs: http://127.0.0.1:8000/docs
# Preserved UI: streamlit run streamlit_app.py
```

11. Conclusion

The Assignment I prototype has been evolved into a feature-preserving, production-style ML system. The final package demonstrates explicit research-to-production separation, modular architecture, persistent artifacts, structured logging, robust validation, a typed REST API, multi-level automated testing, quality gates, privacy-aware audit history, and production experimentation planning.

Appendix A - Canonical submission artifacts

The following files form the canonical submission. Earlier draft artifacts remain only as working history and are not required for grading.

Artifact	Purpose
179.ipynb	Canonical implementation notebook
179_executed.ipynb	Notebook with verified outputs
179.docx and 179.pdf	Submission report
app/, ml/, quality/, services/	Production source package
streamlit_app.py	Preserved Assignment I recruiter workflow
tests/	35 automated tests
reports/	Pytest, lint, and metric evidence
models/resume_classifier.joblib	Trained model artifact
README.md and requirements.txt	Setup and dependencies

Final submission checklist

Check	Verified result
Canonical notebook name	179.ipynb
Notebook execution	179_executed.ipynb completes successfully
Automated tests	35 passed
Model/data quality gates	All required gates pass
Code quality	Black, isort, and Flake8 pass
Report render	DOCX and PDF visually inspected page by page